Technical Report: Assignment 8, Optimize Anything

All ten tracks (A–J) were completed in a **reduced 5-day scope** instead of 14 days (see `plan.md`, "Scope"): every track has an objective, a baseline, a fair comparison, a correctness check and results tables, but sizes are small and some optional extensions were skipped (listed per track under *Limitations*). Every number in this report comes from a file in `results/`; each track's README has the full tables and how to reproduce them.

- **Hardware:** Apple M4 (10 cores: 4 performance + 6 efficiency), 16 GB RAM, macOS 26.6.2. Python 3.12.
- **Labels:** quantum results are **simulated** (no quantum hardware was used). Parallel timings in Track D are **measured** on the M4. FedAvg and multi-task runs are simulated on one machine. xv6 results are measured in QEMU.
- **Reproduce:** `python trackX_*/run.py` per track; `pytest` runs the tests for all tracks.

A. Continuous optimization (README, `results/trackA_continuous/`)

Objective: compare GD, SGD, mini-batch SGD, Momentum, NAG, AdaGrad, RMSProp, Adam, Newton and Gauss–Newton on convex and non-convex objectives. **Baseline:** plain GD. Every method gets the same start, budget, tolerance and learning-rate grid.

result	evidence
Ill-conditioned quadratic (κ = 100): GD 1859 iterations, NAG 246, AdaGrad 20, Newton 1	`a1_landscapes.csv`
Rosenbrock: Momentum 1277, Adam 1230, Newton 6; GD not converged in 5000	`a1_landscapes.csv`
Saddle function: **Newton converges to the saddle (0, 0)**; every first-order method reaches a minimum	`a1_saddle.png`
Noisy gradients (10 epochs, 5 seeds): mini-batch Adam 5.0e-5 ± 9.6e-6; full-batch GD 0.254	`a1_stochastic.csv`
Hand-written backprop = PyTorch autograd to 2.8e-17; HVP = H·v to 2e-15	`a2_*.csv`
Convex O(1/k) (slope –0.84) vs strongly convex linear rate (ρ = 0.975)	`a2_convergence.png`

Trade-off: second-order methods converge in a few steps but need the Hessian and are attracted to saddle points. Diagonal preconditioning reduced the condition number from 10,597 to 1,540, but GD still converged faster. Eigen-analysis explained why: the error had no component along the slowest directions. **Correctness:** finite-difference gradient and Hessian checks, plus a check against SciPy.

B. Discrete optimization (README)

Objective: compare greedy, DP, Branch-and-Bound and ILP; formulate QUBO/Ising models with penalty terms. **Baseline:** greedy (and brute force for verification).

result	evidence
Knapsack: DP = B&B = ILP optimum always; greedy gap ≤ 2.74% (n = 10), 0.10% mean at n = 200	`b1_knapsack_summary.csv`
B&B explores ~760 nodes at n = 200; ILP ~20 ms, mostly CBC start-up	same
Feature selection: greedy 6–16× cheaper, **misses the optimum at k = 3** (0.939 vs 0.950)	`b1_feature_selection.csv`
Max-Cut QUBO = –cut = Ising energy on all bitstrings; SA ratio 1.00 for n ≤ 12	`b2_maxcut.csv`
Penalty too large (P ≥ 5) drops SA success to 0–5%	`b2_feature_qubo_penalty.csv`

C. Quantum optimization (README), all simulated

result	evidence
QAOA Max-Cut ratio 0.778 → 0.851 → 0.918 (p = 1, 2, 3; 6 nodes); simulated annealing gets 1.00	`c1_qaoa.csv`
Noise erases the gain from depth: 8 nodes, p = 1/2/3 → 0.787 / 0.810 / 0.811	`c1_noise.csv`
Shot noise follows 1/√shots (std 0.164 / 0.047 / 0.011)	`c1_shots.csv`
VQE H_2: exact (4e-8 Ha) ideal; **28.6 mHa error with noise** (chemical accuracy 1.6 mHa)	`c1_vqe.csv`
Parameter-shift = autodiff to 3.5e-16; SPSA loss 0.239 vs 0.200 at equal circuit budget	`c2_classifier.csv`
QNG vs GD: tie at 6 steps once both step sizes are tuned	`c2_qng.csv`
Barren plateau: gradient variance 0.087 → 0.0022 (2 → 10 qubits, deep circuits)	`c2_barren_plateau.png`
K_{36} fails to embed on Chimera C8 (512 qubits); Pegasus needs 166 qubits	`c2_embedding.csv`

D. Submodular, parallel and distributed (README)

result	evidence	label
Coverage objective: 0 diminishing-returns violations, greedy ratio 1.00. Pair-bonus objective: 14.3% violations, ratio 0.61 (below 1 − 1/e)	`d1_submodular.csv`	measured
120 uneven SHA-256 tasks, 10 threads: static 5.51×, LPT 6.80×, central queue 6.81×, work stealing 6.71×	`d1_scheduling.csv`	measured
Amdahl serial fraction ≈ 0.05 (LPT); Gustafson scaled speedup 6.96 at p = 10	`d1_gustafson.csv`	measured
20,000 tiny tasks: 0.37–0.50× (slower than serial) at 4–10 threads	`d1_overhead.csv`	measured
Lock contention 69.6 vs 13.7 ms; column-wise access 2.6× slower than row-wise	`d1_sync_locality.csv`	measured
Pipeline bubble 50% → 3% with 32 micro-batches	`parallelism_demo.py` output	simulated
FedAvg: E = 1/5/20 local epochs → 34/7/2 rounds to 98% of centralized; non-IID slower	`d2_fedavg.csv`	simulated
Multi-task: normalized MGDA best on both tasks (MSE 69.7, accuracy 0.979); raw MGDA neglects the large-loss task	`d2_multitask.csv`	measured (CPU)

E. Main case study: exam timetabling (README)

Process: placing each course's exam in a slot and a room so that no student has two exams at once. The baseline mimics a manual registry: first-fit in course-list order. **Data:** seeded synthetic enrolments, 20 and 40 courses (300 / 600 students), 4 rooms, 3 slots per day. **Formulation:**

- variables: binary x[c,t,r] (course in slot and room), y[t] (slot is in the period), b[c,d,t] (back-to-back pairs)
- minimize 20·slots + student back-to-backs
- hard constraints: every course once, no student clash, room capacity, one exam per room per slot

Methods: first-fit (baseline), DSATUR, DSATUR b2b-aware, DSATUR + local search, ILP (CBC, warm start, time limit).

40 courses, 3 seeds (`e_summary.csv`)	slots	back-to-backs	objective	runtime
first-fit (baseline)	20.7 ± 2.1	183 ± 65	596.7 ± 52.2	1 ms
DSATUR	19.7 ± 0.6	324 ± 161	717.0 ± 152.2 (worse)	1 ms
DSATUR b2b + local search	20.3 ± 0.6	14 ± 6	420.7 ± 12.0 (–29%)	0.44 s
ILP (120 s limit)	19.3 ± 0.6 = lower bound	16 ± 7	**403.0 ± 4.4 (–32%)**	115 s

At 20 courses, the ILP objective is 262.0 vs the baseline's 421.7 (–38%).

Findings:

- Plain DSATUR packs courses tightly and creates more back-to-backs. It doesn't shorten the period either, because rooms are the binding constraint (room bound 19–20 slots vs largest clique 9–11).

- Local search does most of the work in under a second.
- The ILP never proved optimality within the limit, but its period matched the lower bound on all 40-course seeds.
- At 160 courses only the heuristics are practical (local search 11.7 s, 152 → 0 back-to-backs).

Correctness: an independent checker validates all hard constraints from the raw enrolment lists (0 violations everywhere), and the ILP equals brute force on tiny instances. The strategy-selection table is in the track README, §5.

F. Compiler optimization (README, code in `trackF_compiler/`)

Built: lexer, parser, three-address IR, CFG + liveness, interpreter, and **6 passes** (constant folding/propagation, CSE, strength reduction, DCE, LICM, copy coalescing). **Baseline:** unoptimized (O0) IR, run on the same interpreter.

result	evidence
Executed instructions −11% (primes) to −47% (sum_squares) over 8 programs; same output on all	`f1_00_vs_opt.csv`
Passes enable each other: folding, CSE and strength reduction give 0 change alone	`f1_pass_ablation.csv`
Register allocation: greedy colouring = ILP optimum in 30/32 cases (0.02 ms vs 14–53 ms)	`f2_regalloc.csv`
Pass ordering: exhaustive best 44,097 vs original order 44,109; **greedy order construction picks the worst (44,697)**	`f2_pass_ordering.csv`

Correctness: differential fuzzing found 0 mismatches in 2,000 random programs. The fuzzer was checked by planting two bugs, which it caught in 49% and 35% of programs.

G. xv6 operating system (README, patch `trackG_xv6/mlfq_scheduler.patch`)

Implemented: a 3-level MLFQ scheduler (slices 1/2/4 ticks, demotion, preemption by higher levels, a priority boost every 40 ticks) plus a `waitstat` system call that returns each child's turnaround, response and run ticks. **Baseline:** stock round-robin (RR). Measured in QEMU.

workload (6 interleaved repeats, 1 CPU)	RR	MLFQ
interactive jobs' turnaround (mixed workload)	33.0 ticks	**13.0**
CPU-bound jobs' turnaround	109.8	110.1
short jobs behind a long one	**22.4**	26.0 (worse)
4 equal jobs (fairness, Jain index)	0.9999	0.9991

Longer slices (2/4/8) help short jobs (23.0) but double interactive response time (3 → 6 ticks), the classic trade-off. **Correctness:** `usertests -q` ALL TESTS PASSED under both schedulers on 3 CPUs; both build with `-Werror`.

H. DBMS: index selection (README, plans in `results/trackH_dbms/h_query_plans.txt`)

Setup: SQLite, TPC-H scale factor 0.1, 8 queries, 13 candidate indexes. **Baseline:** SQLite's default optimizer with no indexes: 1,006 ms. **Formulation:** 0/1 knapsack (benefit vs size under a storage budget), solved with greedy, DP and ILP.

budget	knapsack (greedy = DP = ILP unless noted)	measured greedy
10% (5 MB)	1.08×	**1.36×**
25% (13 MB)	**2.20×**	2.18×
50% (25 MB)	greedy 3.11×, DP = ILP 3.04×	2.97×
all 13	4.93×	

Finding: benefits don't add up. `orders(o_orderdate)` alone makes Q10 hit the 5 s cap (normally 0.17 s), but it becomes useful once `lineitem(l_orderkey)` also exists. **Correctness:** every query returns an identical result hash in every configuration.

I. Quantum algorithms and QML (README), all simulated

result	evidence
DJ and BV: 1 query, 100% ideal; BV 85–90% with noise	`i1_algorithms.csv`
Grover: 25 vs 528 queries at N = 1,024, but the simulation is 6,000× slower than a classical search; 9% success at 5 qubits with noise	`i1_grover_vs_linear.csv`
QFT = DFT matrix to 3.8e-15; QPE exact for ϕ = 1/8	`i1_algorithms.csv`
Shor N = 15: factors 3 × 5 from 51.6% of shots; 13,328 CX gates	`i1_shor.csv`
Annealing: P(optimum) 0.04 → 0.99 as anneal time 0.5 → 50	`i1_annealing.csv`
QML (5 splits): VQC 0.958 ± 0.013, quantum kernel 0.934 ± 0.021, RBF SVM **0.962 ± 0.019**	`i2_qml_vs_classical.csv`

J. Software engineering: test-suite minimization (README)

Target: python-tabulate (2,900-line module, 6,078 lines of tests, 322 passing tests), pinned commit. **Baseline:** the full suite takes 0.807 s inside tests and 1.060 s of wall time. Profiling showed 10 subprocess CLI tests taking 75% of the time, plus one-time pandas/numpy imports. **Formulation:** keep test t ($x_t \in \{0,1\}$) to minimize predicted run time, such that each of the 938 covered lines stays covered. Heavy imports are a fixed charge, $z_m \geq x_t$ (a fixed-charge weighted set cover).

suite (`j_selection.csv`, `j_mutation.csv`)	tests	time in tests	wall (median of 5)	line coverage	mutants killed / 40
full suite	322	0.807 s	1.060 s	100%	33
greedy	70	0.247 s	0.427 s	100%	31
ILP (proven optimal, 0.14 s solve)	**68**	**0.245 s (–70%)**	**0.425 s (2.5×)**	**100%**	30
random, same size	68	0.122 s	0.278 s	76.8%	27

Findings:

- Greedy is within 0.1% of the ILP.
- The cost model mattered more than the solver. A naive model (raw per-test times, in-process coverage) dropped every CLI test, and predicted 2.5× too little time, because imports are charged to whichever test runs first (failure log #19, #20).
- **The cost in correctness:** the minimized suite misses 3 of the 33 mutants the full suite catches, even at 100% line coverage. Use it as a fast CI path and keep the full suite nightly.

Answers to the 10 final questions

1. What process did you choose, and what evidence showed that it needed optimization? Main case study (E): exam timetabling.

- The list-order baseline left 175–183 students with back-to-back exams per instance.
- It used up to 3 more slots than the provable lower bound (23 vs 20 on 40 courses, seed 2).

Each fixed track also started from a measured bottleneck:

- J: CLI subprocess tests were 75% of tabulate's test time.
- H: SQLite took 1,006 ms with no indexes.
- G: interactive jobs waited 33 ticks behind CPU-bound jobs under round-robin.
- F: unoptimized IR executed up to 1.9× more instructions (sum_squares 4,508 vs 2,407).
- D: a static split left 38% load imbalance.

2. How did you formulate the objective function and constraints? Every track's README has the variables, objective and constraints.

- E: binary x[c,t,r], y[t]; minimize 20·slots + back-to-backs; hard constraints for clashes, room capacity and one exam per room per slot.
- J: weighted set cover with fixed import charges.
- H: 0/1 knapsack (benefit vs index size under a storage budget).
- F: register allocation as an ILP (minimize spill cost, interfering variables get different registers); pass ordering as a search over permutations.
- B/C: knapsack, and Max-Cut / feature selection as QUBO with penalty terms.
- A: unconstrained losses with L2 regularization.
- D: makespan; MGDA's min-norm problem.

3. Which Chapter 6 methods did you test, and why were they appropriate? All of them, mapped in `docs/chapter6_checklist.md`.

- Continuous methods (A) for differentiable losses.
- Exact discrete methods (DP, B&B, ILP) where instances are small enough to prove optimality (B, E, F, H, J), and greedy / DSATUR / local search where they must scale (E at 160 courses, J, H).
- QUBO / QAOA / annealing (B, C, I) on binary problems small enough to simulate.
- Submodular greedy only after testing diminishing returns (D).
- Load-balancing schedulers for uneven parallel tasks (D).
- FedAvg / MGDA / GradNorm for distributed and multi-task training (D).

Methods that don't fit a problem are stated as such. For example, continuous methods are not used for timetabling, and a QUBO version of E would hit the penalty and embedding limits shown in B and C.

4. How did the optimized solution compare with the baseline?

- **E:** objective –32% (40 courses) and –38% (20 courses); back-to-backs –87% to –92%; 0 violations.
- **J:** 2.5× faster wall time and –70% test time, keeping 100% line coverage.
- **H:** 2.20× at a 25% storage budget, 4.93× with all indexes.
- **G:** interactive turnaround 33 → 13 ticks.
- **F:** –11% to –47% executed instructions.
- **D:** 5.51× → 6.80× speedup with load balancing.
- **C/I:** quantum methods did **not** beat the classical baselines. SA solves Max-Cut exactly; RBF SVM 0.962 vs VQC 0.958.

5. What trade-offs, failures or limitations did you discover?

- **Speed vs proof:** the ILP needs 115 s against 0.44 s for local search for 4% better (E); greedy is almost free and near-optimal in B, H and J.
- **Models vs reality:** knapsack benefits don't add up (H: one index made Q10 slower than the 5 s cap); test costs depend on execution order (J); greedy pass ordering picked the worst order (F).
- **Policy trade-offs:** MLFQ hurts short jobs that don't fit the top queues (G); line coverage doesn't preserve fault detection (J: 3 of 33 mutants missed).
- **Quantum and parallel limits:** noise erases QAOA depth gains; barren plateaus; tiny parallel tasks run slower than serial.
- **AI errors:** 20 AI-generated errors are documented in `docs/failure_log.md`.
- **Scope:** sizes are small and run on one laptop; quantum results are simulated only; the scope was cut from 14 to 5 days.

6. What evidence shows that your results are correct and reproducible?

- 96 pytest tests, all passing. They include exact methods equal to brute force (B, E, F, H, J), gradients checked against finite differences and autodiff (A, C), and quantum circuits checked against analytic results (I).
- Independent checkers: the timetable checker (E), register-allocation validity (F), and query result hashes identical in every configuration (H).
- Differential fuzzing of the compiler: 0 mismatches in 2,000 programs, and planted bugs were caught.
- `usertests` passes under both schedulers (G). Reduced test suites were re-run to measure coverage (J).

- Fixed seeds everywhere; repeated runs with mean ± std for stochastic methods; interleaved A/B runs for timing (G, J).
- Every number in this report comes from a CSV in `results/`, and one `run.py` per track regenerates them.

7. How could the approach be adapted if the scale, data, constraints or hardware changed?

- **E:** at hundreds of courses, drop the ILP and use DSATUR + local search, which handles 160 courses in 12 s; parallel restarts are trivial. Keep the ILP for bounds or sub-problems. New soft constraints only add objective terms. Real enrolment data would replace the generator without code changes.
- **J:** on a big suite, the ILP stays easy (set cover with merged constraints), but profiling cost grows. Coverage could be profiled incrementally per changed file, and branch or mutation data could replace line coverage when fault detection matters more.
- **H:** with more candidates, use measured or interaction-aware selection rather than an additive model.
- **D:** on a GPU or cluster, communication becomes the bottleneck, so prefer FedAvg-style local steps.
- **C/I:** on real quantum hardware, keep circuits shallow because of noise.

8. Compiler, xv6 and DBMS: what was implemented, what concept was learned, and how was correctness preserved?

- **F:** 6 passes on a three-address IR (constant folding/propagation, CSE, strength reduction, DCE, LICM, coalescing), plus greedy vs ILP register allocation and pass-order search.
  - Concept: dataflow analysis (liveness), passes that enable each other, phase ordering.
  - Correctness: identical output on all programs, and 0 mismatches in 2,000 fuzzed programs, using a fuzzer shown to catch planted bugs.
- **G:** a 3-level MLFQ scheduler and a `waitstat` system call in xv6-riscv.
  - Concept: scheduling policy vs mechanism, lock ordering in `proc.c` (never two process locks at once), starvation prevention by priority boost.
  - Correctness: `usertests -q` passes under both schedulers on 3 CPUs; builds with `-Werror`.
- **H:** index selection on TPC-H in SQLite.
  - Concept: access paths and join order. A new index changes the planner's choices, sometimes for the worse (`h_query_plans.txt`).
  - Correctness: identical result hashes for every query in every configuration.

9. What advantages and limits of QML, quantum algorithms and parallel computing did your experiments reveal?

- **Quantum (all simulated):**
  - Grover needs fewer queries (25 vs 528 at N = 1,024), but simulating it is 6,000× slower than a classical search.
  - DJ and BV need 1 query, and Shor factors 15 (51.6% of shots) but needs 13,328 CX gates.
  - Noise drops Grover to 9% at 5 qubits and puts VQE 28.6 mHa off.
  - Barren plateaus shrink gradient variance 40× from 2 to 10 qubits.
  - QML ties but doesn't beat a tuned RBF SVM, at a far higher circuit cost.
- **Parallel (measured):** a 6.8× speedup on 10 cores today, limited by Amdahl (serial fraction ≈ 5%), overhead (tiny tasks 0.37×), locks and locality.

Parallelism gives a bounded constant factor that is available now. Quantum algorithms promise better scaling that today's noise and simulation costs don't deliver.

10. Which AI tools did you use, how did their outputs differ, and which AI-generated errors did you find and fix?

- **Claude Code (Claude Opus 5.5)** was used for all tracks (register in the README; prompts per track in `prompts/`; commits tagged `[AI:claude-code]`).
- **Second tool:** prompt B1 was also given to **Claude Sonnet 5.5**, as a separate agent with a fresh context and an isolated folder (`docs/tool_comparison.md`, output in `prompts/B/tool2_sonnet_B1/`).

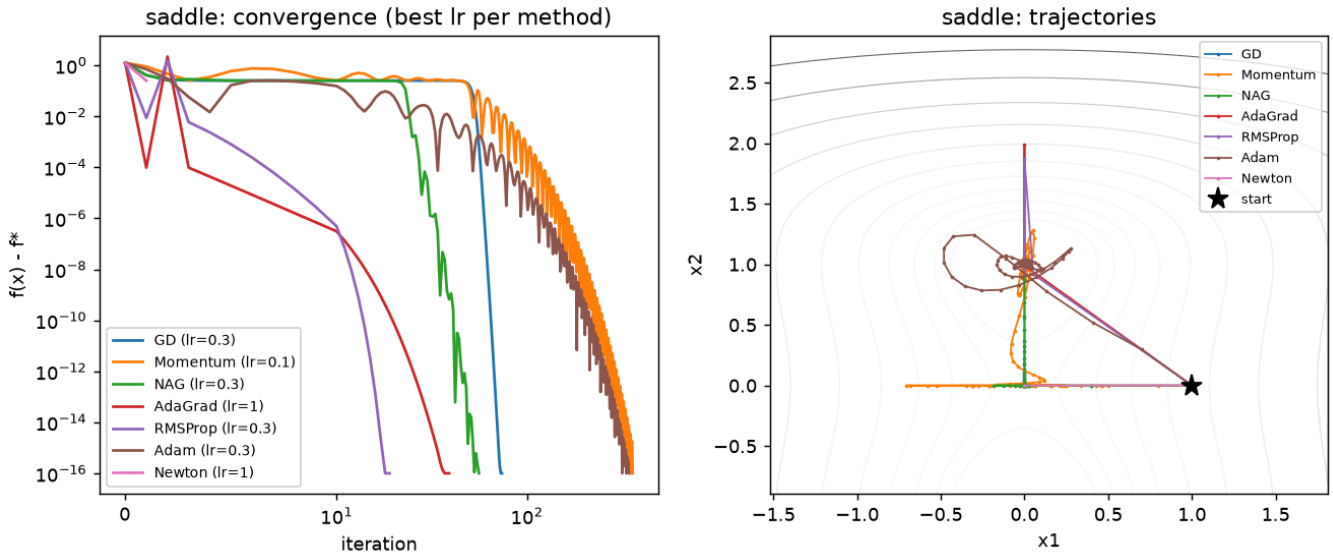
- Sonnet was correct on the first try (20/20 tests in 47 s; its optima match ours on all 12 instances), but it returned only objective values and never checked feasibility.
- Opus did more verification (an independent `check()`, harder instances), but needed a fix for an out-of-date PuLP API.
- Both are Claude models, so a non-Claude tool would make a stronger comparison.

- **Errors found and fixed:** 20, in `docs/failure_log.md`. They include:
  - unfair comparisons: coarse or edge-of-grid hyperparameter tuning, untuned quantum-kernel baselines
  - wrong evaluation points: Fisher at w = 0
  - workloads that measured the wrong thing: matmul on the AMX coprocessor, non-interleaved QEMU runs
  - method bugs: MGDA scaling the task heads
  - optimizer and model errors: phase ordering made code worse; additive index benefits hung the DBMS run; test-minimization costs that depended on execution order, plus a coverage blind spot Most were caught because a result was implausible, and the error was confirmed with a targeted check before fixing.

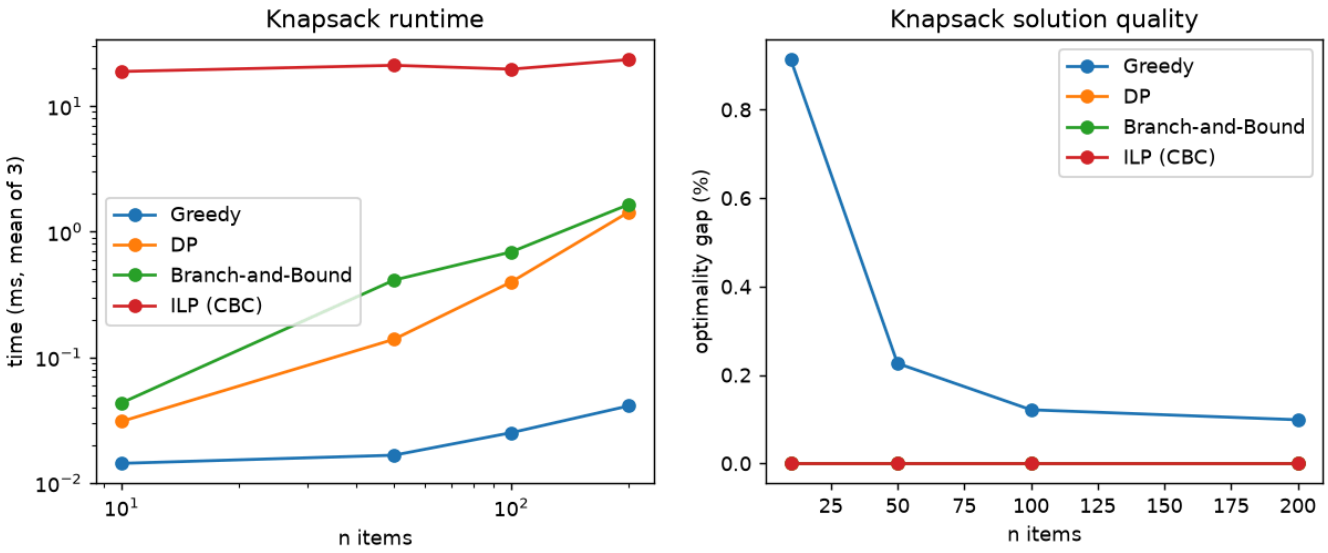
Review notes (prompt V1: skeptical self-review)

- **Re-run for this report:** all 96 tests (7 s). Every `run.py` was run when its track was built (the E and J runs on 2026-10-30). Tracks G (needs QEMU) and H (≈ 5 min) were not re-run from a fresh venv.
- **Variance:** stochastic results report repeats (A 5 seeds, C 3–5 seeds, D 3 runs, E 3 instances, I 5 splits, J 5 timing repeats). Single-run items: B penalty sweep (20 SA seeds, success rate), F (deterministic).
- **Not proven optimal:** the Track E ILP (time limit). Its period is optimal because it equals the lower bound; the back-to-back count is not proven.
- **Speedup claims** are measured on one laptop, with mixed P/E cores (D). Quantum "speedups" are reported only as query counts, never as wall-clock gains.
- **Missing from the 14-day brief:** real quantum hardware, PostgreSQL / join-order selection (H2), a second xv6 feature, and LLVM. These were left out deliberately to fit the 5-day scope.

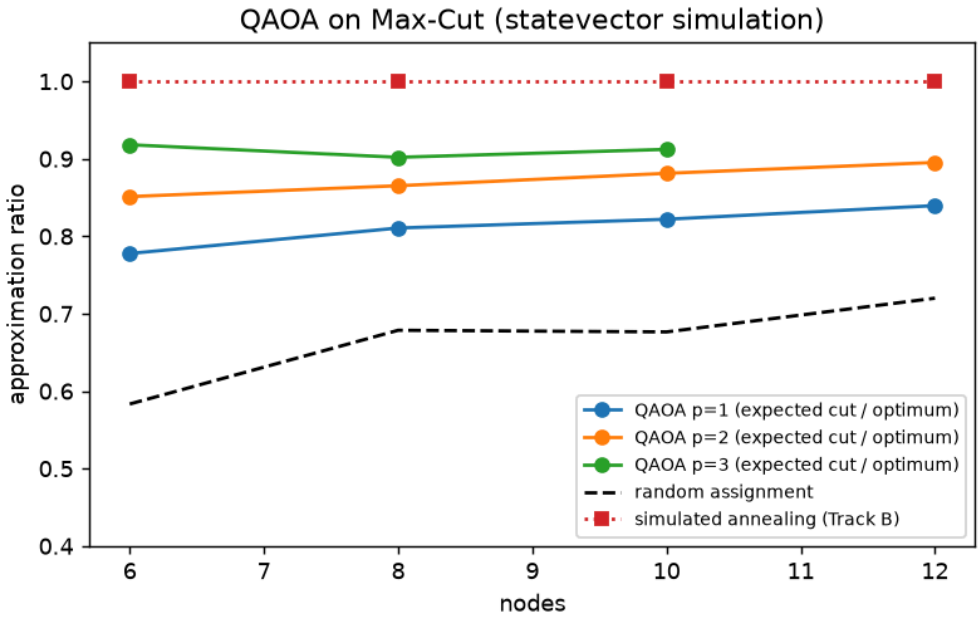
Appendix: key plots



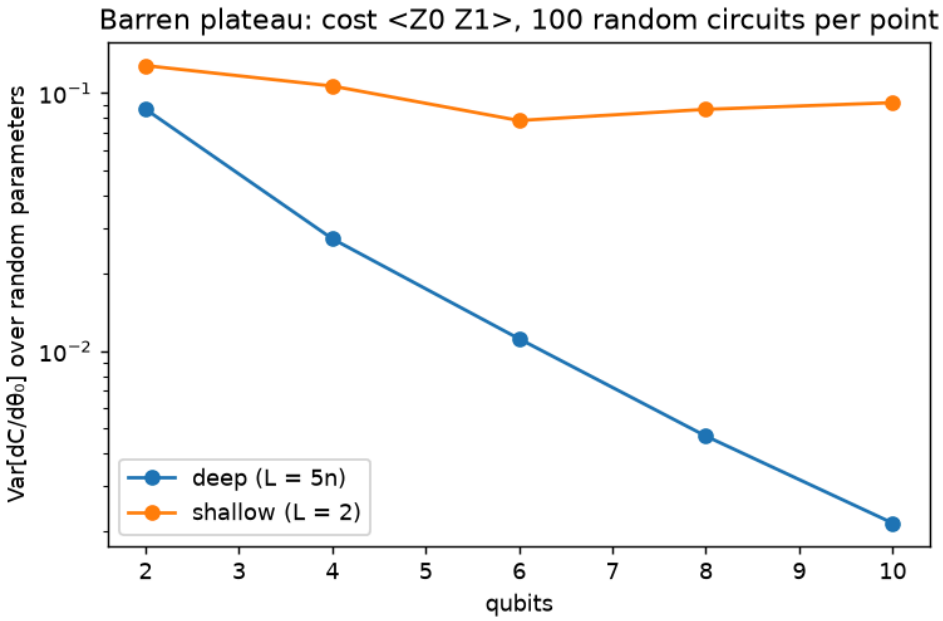
Track A: Optimizer paths on the saddle function: Newton converges to the saddle point.
(`results/trackA_continuous/a1_saddle.png`)



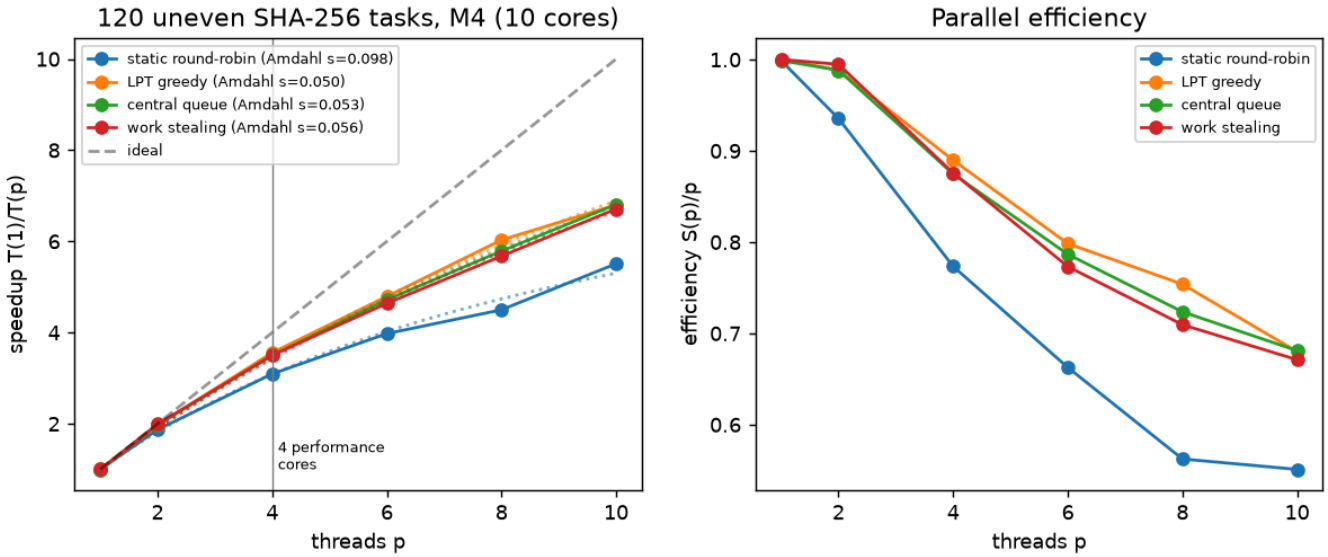
Track B: Knapsack runtime and greedy gap vs n. (`results/trackB_discrete/b1_knapsack.png`)



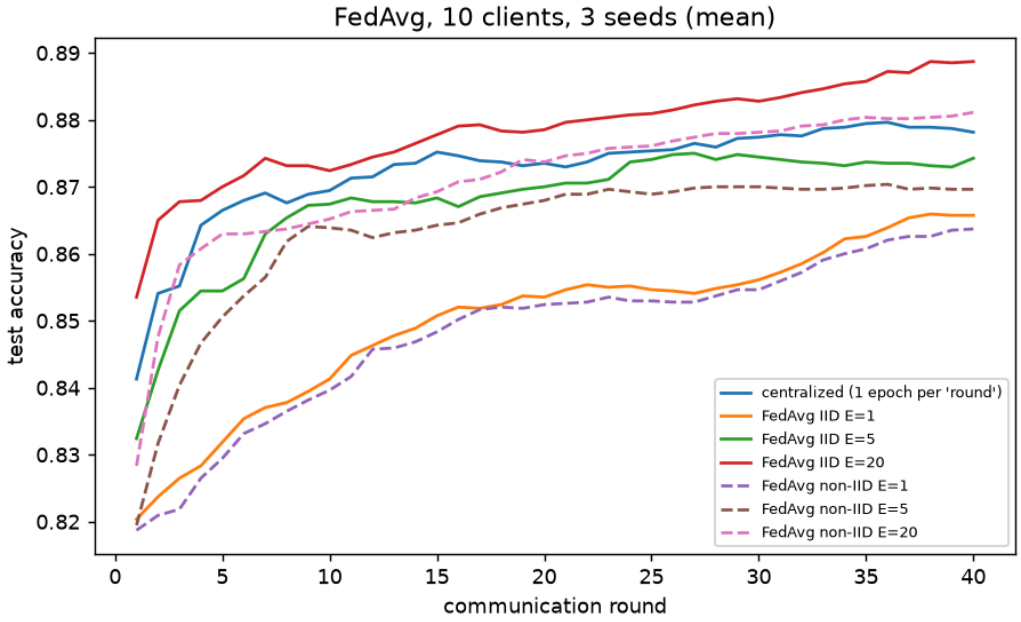
Track C: QAOA approximation ratio vs depth p (simulated). (`results/trackC_quantum_opt/c1_qaoa.png`)



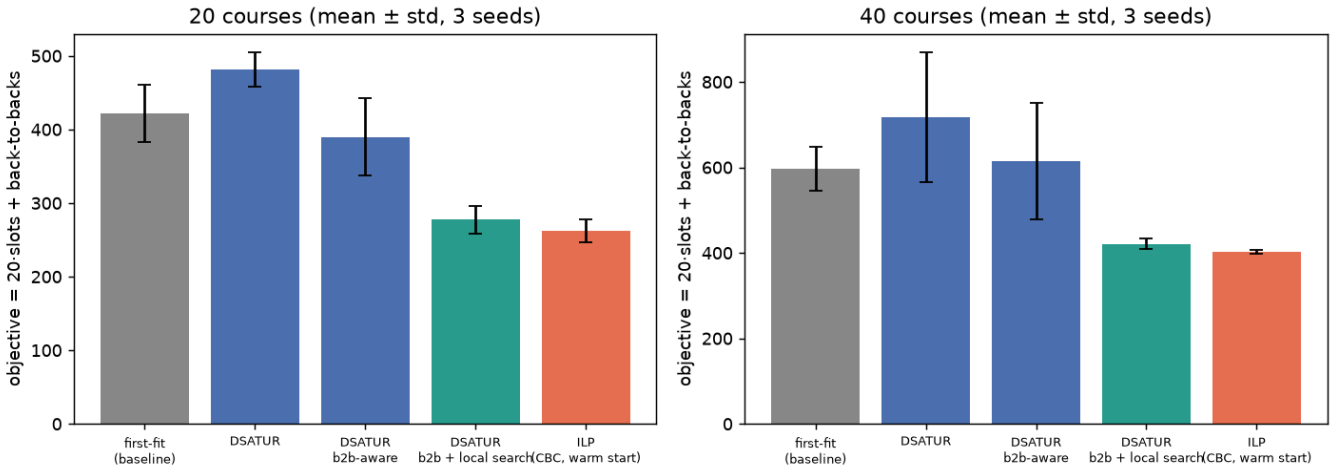
Track C: Barren plateau: gradient variance vs number of qubits (simulated).
(`results/trackC_quantum_opt/c2_barren_plateau.png`)



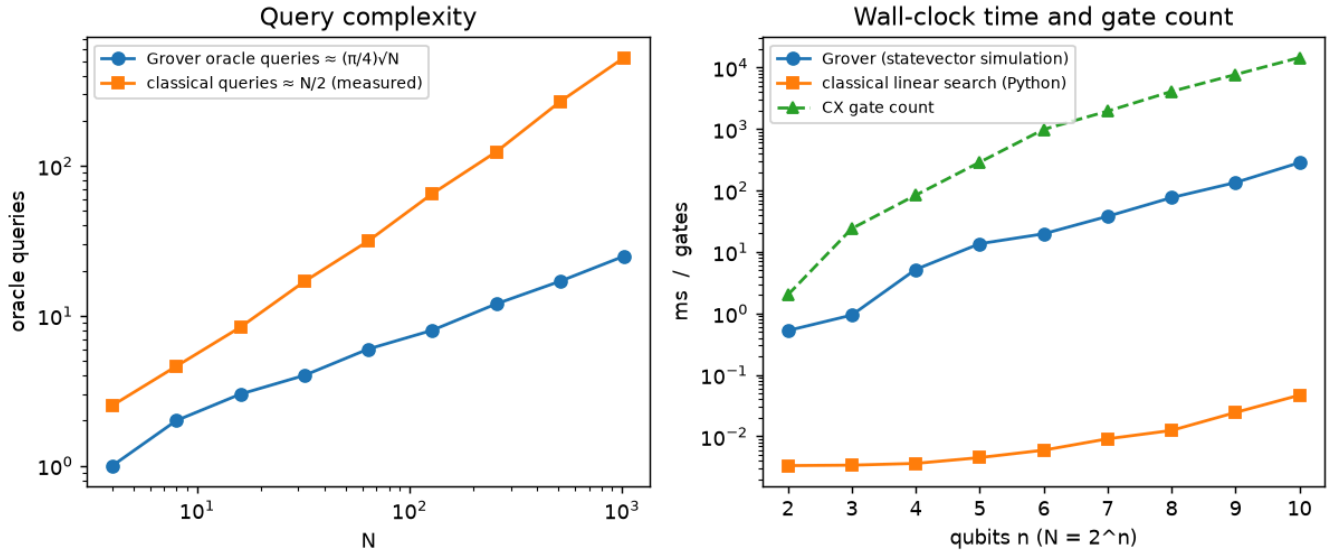
Track D: Parallel speedup per scheduling strategy (measured, M4).
(`results/trackD_submodular_parallel_distributed/d1_scheduling.png`)



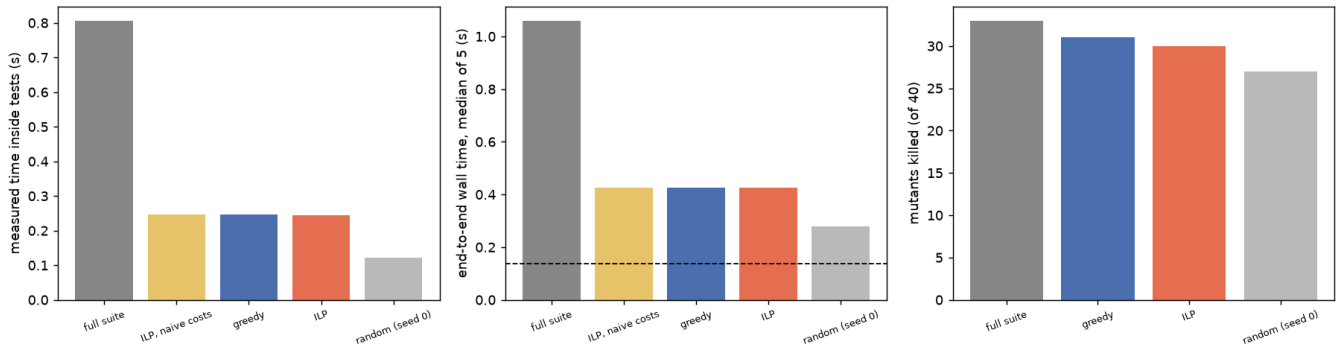
Track D: FedAvg vs centralized training (simulated).
(`results/trackD_submodular_parallel_distributed/d2_fedavg.png`)



Track E: Exam timetabling objective per method (mean ± std, 3 seeds). (`results/trackE_case_study/e_objective.png`)



Track I: Grover vs linear search: oracle queries and simulation time.
(`results/trackI_quantum_algos_qml/i1_grover_vs_linear.png`)



Track J: Test-suite minimization: test time, wall time, mutants killed. (`results/trackJ_software_eng/j_minimization.png`)